

Threading & Concurrency (.NET / OS Concepts)

DAY - 1

1	Foundations	Process vs thread vs AppDomain; Why threading (responsiveness, parallelism, throughput); Thread lifecycle and states; UI/main thread concept; Foreground vs background threads; Context-switching cost; CPU cores vs threads	Conceptual session. No heavy code. Use diagrams.
---	-------------	--	--

1. Process

Definition: A Process is a running instance of a program with its own memory space, resources and execution context.

Program -> OS loads executable -> Process created -> Main Thread created -> Execution begins.

Memory Layout:

- Code/Text Segment: instructions, functions
- Data Segment: global/static variables
- Heap: dynamic memory
- Stack: local variables, function frames

Key points:

- Every process has at least one thread.
- Processes have isolated memory spaces.
- IPC examples: pipes, sockets, shared memory.

2. Thread

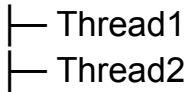
Definition: Smallest unit of CPU execution inside a process.

Shared Resources: Code, Heap, Data Segment, Files, Address Space.

Private Resources: Stack, Registers, Program Counter.

Hierarchy:

Process



Thread advantages:

- Responsiveness
- Concurrency
- Better CPU utilization
- Parallel execution on multi-core CPUs

3. AppDomain (.NET)

Logical isolation boundary inside a .NET process (CLR construct).

Hierarchy:

OS -> Process -> AppDomain -> Threads

Default: Every .NET process starts with one Default AppDomain.

Useful APIs:

`AppDomain.CurrentDomain.FriendlyName`

`AppDomain.CreateDomain()`

`CreateInstanceAndUnwrap()`

`MarshalByRefObject`: enables cross-AppDomain communication by reference (proxy).

Modern Note: .NET Core / .NET 5+ mainly uses `AssemblyLoadContext`.

4. Why Threading?

Responsiveness: Keep application interactive during long work.

Parallelism: True simultaneous computation using multiple CPU cores.

Concurrency: Manage multiple tasks during same time period.

Throughput: Amount of work completed per unit time.

Common APIs:

Responsiveness -> `Task.Run`, `async/await`, `Thread`

Parallelism -> `Parallel.For`, `Parallel.ForEach`, `AsParallel`

Concurrency -> `Task.WhenAll`, `Thread`, `async/await`

Throughput -> `ThreadPool`, batching, `async I/O`

5. Thread Lifecycle & States

States: (thread.ThreadState)

- Unstarted: Created but Start() not called
- Runnable/Ready: Waiting for scheduler
- Running: Currently executing
- WaitSleepJoin: Sleep/Join/Wait state
- Stopped: Finished execution

Lifecycle:

Unstarted -> Start()

Ready -> Running

Running -> WaitSleepJoin

WaitSleepJoin -> Ready

Running -> Stopped

Join(): Blocks the calling thread until target thread completes.

6. UI/Main Thread

UI Thread handles:

- Button clicks
- Mouse/keyboard events
- Rendering/redrawing

Blocking UI thread causes freezes.

Fix: Use Task.Run / async-await to move heavy work off UI thread.

7. Foreground vs Background Threads

Foreground Thread:

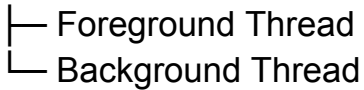
- Keeps process alive
- Important work

Background Thread:

- Does not keep process alive
- Killed when all foreground threads finish

Important:

Thread



Background thread != ThreadPool thread

Default new Thread() => Foreground

8. ThreadPool

Managed pool of reusable worker threads maintained by CLR.

Purpose:

- Avoid expensive thread creation/destruction
- Improve throughput

Used by:

Task.Run

Parallel.For

async/await (often indirectly)

9. Context Switching

Definition: Saving one execution state and loading another.

Context contains:

- Registers
- Stack Pointer
- Program Counter
- Execution State

Thread switch: cheaper (shared memory)

Process switch: costlier (separate address spaces, page tables)

Too many threads increase switching overhead.

10. CPU Cores vs Threads

CPU Core: Physical execution unit.

CPU Thread / Logical Processor: Logical execution context exposed by a core (Hyper-Threading/SMT).

Important Distinction:

1. Software Thread (.NET new Thread())
2. OS Thread (scheduler-managed)
3. Hardware Thread / Logical Processor (CPU capability)

Execution Flow:

Your App

- > Software Threads
- > OS Scheduler
- > CPU Logical Processors
- > Physical CPU Cores

Example:

8 cores, 16 logical processors.

100 software threads != 100 simultaneous executions.

11. Quick Notes

- Process always has at least one thread.
- Threads share process memory; processes do not.
- Concurrency != Parallelism.
- Parallelism needs multiple cores.
- Background threads do not keep process alive.
- ThreadPool improves efficiency through reuse.
- Process context switch > Thread context switch cost.

DAY - 2

2	Low-level Thread API	<code>System.Threading.Thread</code> (Start, Join, Sleep); Passing parameters; <code>ParameterizedThreadStart</code> ; Why <code>Thread.Abort</code> is deprecated; <code>ThreadPool.QueueUserWorkItem</code> ; When (not) to use <code>Thread</code> directly	Teach as historical baseline. Most modern code uses <code>Task</code> .
---	----------------------	--	---

System.Threading.Thread is low-level class used to create and manage an operating system thread.

1. **Thread.Start()** - **Purpose:** Instructs the operating system to begin executing the thread.

new Thread() — Unstarted state — Memory is allocated — thread.start() — Runnable state waits in threadpool until the OS thread scheduler allocate CPU time

2. **Thread.Sleep** - **Purpose:** Pauses the current executing thread for a specified period.
 - **Thread.Sleep(1000) (Actual Sleep)**
 - **Action:** The thread is completely removed from the CPU execution queue.
 - **State Change:** Transitions from Running to the **WaitSleepJoin** state.
 - **OS View:** The OS scheduler marks this thread as *unavailable* for the duration. It will not run until the timer expires.
 - **Thread.Sleep(0) (The Polite Yield)**
 - **Action:** The thread yields its remaining time slice, saying, "If there is another thread of equal priority waiting, it can skip ahead of me right now. If not, I'll keep running."
 - **State Change:** Stays in the **Runnable (Ready)** state. It never drops into a blocked state.

- **OS View:** If no other threads are waiting, execution continues immediately. If one is waiting, the OS swaps your thread out and puts it at the end of the line.
- **Use Case:** Used in intensive background computing loops (like **SpinLocks**) to ensure the thread doesn't hog the entire CPU core.
- **Modern .NET Tip:** Use **Thread.Yield()**, which is preferred as it allows *any* ready thread to run on that CPU core, even if it has a lower priority.

Thread doing its work — identifies Thread.Sleep() — that particular thread goes to sleep — state WaitSleepJoin — yield its current CPU time slice to other thread in queue

3. Thread.Join() - Purpose: Blocks the calling thread until the target thread finishes execution. Join() is used for synchronization

If Thread A calls ThreadB.Join(), Thread A will stop completely and wait at that line of code until Thread B completes its work and enters the Stopped state.

```
Static void Main(){ //main thread created
    Thread t1 = new Thread(); //main thread is t1 thread
    Thread t2 = new Thread(); //another new thread t2
    Thread t3 = new Thread(); //another new thread t3
    t1.start();
    t2.start();
    t3.start();
    t2.join(); //blocked t1 (main thread) t2 will does it work and unblock t1
    t3.join(); //blocked t1 (main thread) t3 will does it work and unblock t1
}
```

Why use it?

It ensures that dependent tasks happen in the correct sequence. For example, you cannot analyze data (Thread A) until the data-loading thread (Thread B) is completely finished.

Passing parameters to threads

1. No parameter thread
2. ParameterizedThreadStart (only one parameter)
3. Lambda Expression (multiple parameters)
4. Custom object /DTO (complex data transfer, large input payload)
5. Modern .NET approach (using Task.Run(), accept multiple parameter)

No Parameter

```
class Program
{
    static void Main(string[] args)
    {
        Thread t1 = new Thread(Work);
    }

    static void Work()
    {
        Console.WriteLine("No Parameter");
    }
}
```

ParameterizedThreadStart

```
class Program
{
    static void Main(string[] args)
    {
        Thread t1 = new Thread(new ParameterizedThreadStart(Work));
        t1.Start("Hello");
    }

    static void Work(object data)
    {
        Console.WriteLine(data);
    }
}
```


Lambda Expression

```
class Program
{
    static void Main(string[] args)
    {
        Thread t1 = new Thread(()=>
        {
            Work(1, "Samyuktha");
        });
        t1.Start();
    }
    static void Work(int id, string name)
    {
        Console.WriteLine($"{id} - {name}");
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        int x = 10;
        Thread t1 = new Thread(() =>
        {
            Console.WriteLine(x);
        });
        t1.Start();
        x = 20;
    }
}
```

Lambda captures Variable, not the value so the updated value gets printed

Custom object /DTO (Data transfer object)

```
class Program
{
    static void Main(string[] args)
    {
        Thread t1 = new Thread(Work);
        t1.Start(
            new UserData
            {
                Id = 1,
                Name = "Samyuktha"
            });
    }
    static void Work(object obj)
    {
        UserData user = (UserData)obj;
        Console.WriteLine($"{user.Id} - {user.Name}");
    }
}

class UserData
{
    public int Id;
    public string Name;
}
```

Modern .NET approach

```
class Program
{
    static void Main(string[] args)
    {
        Task.Run(() =>
        {
            Work("Hello");
        });
    }
    static void Work(object obj)
    {
        Console.WriteLine(obj);
    }
}
```

Thread.Abort - Forcefully terminates thread execution

Thread running — thread.abort() – forceful stop creates multiple problems — unsafe

Problem due to forceful termination:

1.Resource corruption: File left opened, DB transaction half-done, Socket not closed

2.Shared state corruption:

Threads share:Heap, Objects, Variables

Abort mid-operation:partial updates, inconsistent state

3.Lock problems:

Abort inside: lock(obj) { }

Cause: Deadlocks, Unreleased locks

CancellationToken - For safe cancellation of thread

Cooperative cancellation mechanism used to safely request work cancellation

CancellationTokenSource - Creates & controls cancellation

CancellationToken - Passed to worker.

Cancel() - Request cancellation.

IsCancellationRequested - Worker checks signal.

ThrowIfCancellationRequested() - Shortcut API.

```

class Program
{
    static void Main(string[] args)
    {
        CancellationSource cts = new CancellationSource();
        Task worker = Task.Run(() => Work(cts.Token));
        Console.WriteLine(worker.Status); //waiting to run
        Thread.Sleep(5000);
        cts.Cancel();
        worker.Wait();
        Console.WriteLine(worker.Status); //Ran to Completion

        CancellationSource cts2 = new CancellationSource();
        Thread t1 = new Thread(()=>
        {
            Work(cts2.Token);
        });
        Console.WriteLine(t1.ThreadState); //unstarted
        t1.Start();
        Console.WriteLine(t1.ThreadState); //Running
        Thread.Sleep(5000);
        cts2.Cancel();
        Console.ReadLine();
    }

    static void Work(CancellationSource token)
    {
        while (true)
        {
            if (token.IsCancellationRequested)
            {
                Console.WriteLine("Thread Cancelled");
                break;
            }
            Console.WriteLine("Working.....");
            Thread.Sleep(1000);
        }
    }
}

```

ThreadPool

Thread creation is expensive. Why?

- Allocate stack memory
- Create OS thread object
- Register with scheduler
- Manage lifecycle
- Context switching cost

ThreadPool - Is a CLR-managed pool of reusable worker threads used to execute short background work efficiently.

API used

Without parameter - `ThreadPool.QueueUserWorkItem(Work);`

With parameter - `ThreadPool.QueueUserWorkItem(Work,"Hello");`

To check ThreadPool:

```
class Program
{
    static void Main()
    {
        for(int i = 1; i < 10; i++)
        {
            ThreadPool.QueueUserWorkItem(Work,i);
        }
        Console.ReadLine();
    }

    static void Work(object i)
    {
        Console.WriteLine($"Running {i} on thread - {Thread.CurrentThread.ManagedThreadId}");
        Thread.Sleep(1000);
        Console.WriteLine($"Closed {i} on thread - {Thread.CurrentThread.ManagedThreadId}");
    }
}
```

```
Running 2 on thread - 4
Running 3 on thread - 5
Running 4 on thread - 3
Running 1 on thread - 6
Running 5 on thread - 7
Closed 3 on thread - 5
Closed 2 on thread - 4
Running 7 on thread - 4
Running 6 on thread - 5
Closed 4 on thread - 3
Running 8 on thread - 3
Closed 1 on thread - 6
Running 9 on thread - 6
Closed 5 on thread - 7
Closed 7 on thread - 4
Closed 6 on thread - 5
Closed 8 on thread - 3
Closed 9 on thread - 6
```

Creation of Thread involves:

- OS thread creation
- Stack memory allocation
- Scheduler bookkeeping
- Context switching overhead
- Lifecycle management

For performing small task no new Thread() to be created as it's worst case, instead we use **ThreadPool, async await, Task**.

These are preferred because:

- ThreadPool - Reuses the thread (tiny works)
- Async await - Waits until one thread completes and assigns the task (http request, DB handling)
- Task - Single task keeps on running in Background

DAY - 3

3	Synchronization Primitives	Race conditions and atomicity; <code>lock</code> / <code>Monitor</code> ; Picking a good lock object; <code>Mutex</code> , <code>Semaphore</code> , <code>SemaphoreSlim</code> ; <code>ReaderWriterLockSlim</code> ; <code>Interlocked</code> (Increment, CompareExchange); <code>volatile</code> and memory barriers (briefly); <code>ManualResetEvent(Slim)</code> , <code>AutoResetEvent</code> , <code>CountdownEvent</code> ; Deadlock prevention (lock ordering, timeouts)	The hardest phase. Plan a deadlock- debugging exercise.
---	-----------------------------------	--	---

Synchronization - Coordinating multiple threads so they access shared resources safely and in a controlled manner.

Synchronization Primitives - Low-level mechanisms used to coordinate thread execution and protect shared resources.

Synchronization is achieved through:

Lock, Monitor, Mutex, Semaphore, Interlocked, Volatile, ReaderWriterLockSlim in a different manner.

Synchronization Primitives exists to solve 3 main problems:

1. Race condition - Multiple threads touching same data.
2. Atomicity
3. Ordering / Visibility

Race Condition - Is a concurrency bug, When multiple threads access/modify shared data concurrently and result depends on execution timing/order without proper synchronization.

Why this happens? Threads inside a process have shared data like heap, obj, variables, shared data. When multiple thread touches the same data then all the threads have same heap, obj, variables and data without coordination.

When it occurs? Shared Data, Multiple Threads, Concurrent Access, No Synchronization, Non-Atomic Operations

Eg:

int Balance = 1000;

Thread 1 : Balance -= 100

Thread 2: Balance -= 200

Expected: Balance = 700

Reality: Balance = 800

What happens inside Scheduler execution:

Thread 1

 Reads 1000

Scheduler switches

Thread 2

 Reads 1000

 Calculate 800

 Write 800

Scheduler switches back

Thread 1 resumes

 Calculate 900

 Write 900

Final output: 800 which is wrong

Race Conditions are Non-Deterministic, same code - different runs.

Depends on: **Scheduler timing and Thread execution order**

How to prevent Race Condition? Using Synchronization Primitives like lock, interlocked, mutex, monitor, semaphore etc.,

- Lock -> Mutual Exclusion (Only one thread can execute protected code at a time)

Eg: Lock Program

```
class Program
{
    static object myLock = new object();
    static int Balance = 1000;
    static void Main(string[] args)
    {
        Thread t1 = new Thread(() =>
        {
            Work(100);
        });
        Thread t2 = new Thread(() =>
        {
            Work(200);
        });
        t1.Start();    t2.Start();
        t1.Join();     t2.Join();
        Console.WriteLine($"Final Balance: {Balance}");
    }
    static void Work(int data)
    {
        lock (myLock)
        {
            Balance -= data;
            Console.WriteLine($"Balance: {Balance}");
        }
    }
}
```

Atomic: Operation behaves as one indivisible unit.

- Reads/Writes of bool, byte, char, short, int, float, and references are always atomic.
- 64-bit types (long, double) are atomic only on 64-bit runtimes; they are non-atomic on 32-bit systems.

Non-Atomic: Multi-step operations that can be interrupted.

- Read-Modify-Write operations like i++, i--, i += 5, or x = x + y are NEVER atomic.
- Compound logic (checking a value then updating it) is NEVER atomic without synchronization

Non-Atomic function can be fixed using:

- **Interlocked** = best for arithmetic operation like increment, decrement, exchange, compareExchange(Single atomic operation)
- **Volatile** = used for simple read/write visibility

- **Lock** = used when multiple lines of code or multiple variables must be updated together as a single unit. (Entire critical section)

Interlocked:

1. Interlocked.Increment(ref variableName)
2. Interlocked.Decrement(ref variableName)
3. Interlocked.Exchange(ref oldVariable, newVariable)
4. Interlocked.CompareExchange(ref variableName, newValue, expectedValue)

Monitor: Synchronization primitive that provides mutual exclusion and thread coordination for shared resources.

Lock is a convenient wrapper around Monitor.Enter()/Exit().

```
Lock(myLock){  
    Work();  
}
```

```
Monitor.Enter(myLock);  
try(){  
    Work();  
}  
Finally(){  
    Monitor.Exit(myLock);  
}
```

Monitor API :

Monitor.Enter(lockObj) - Acquire ownership of synchronization object.

Monitor.Exit(lockObj) - Release ownership

Monitor.TryEnter(lockObj, time, condition) - Attempt acquisition with optional timeout

Monitor.Wait(lockObj) - No data so sleep until one arrives

Monitor.Pulse(lockObj) - New work arrived so wake up the waiting thread

Monitor.PulseAll(lockObj) - Wakes up all waiting thread

Volatile: Provides visibility guarantee between threads

```
static volatile bool Stop = false;
static void Work()
{
    while (!Stop)
    {
        Console.WriteLine("Program Working.....");
        Thread.Sleep(500);
    }

    Console.WriteLine("Program Stopped....");
}
static void Main(string[] args)
{
    Thread t1 = new Thread(Work);
    t1.Start();
    Thread.Sleep(5000);
    Console.WriteLine("From main asking work to stop");
    Stop = true;
    t1.Join();
    Console.WriteLine("Work completed");
    Console.ReadLine();
}
```

Mutex(Mutual Exclusion) - Synchronization primitive that provides exclusive access to a resource, including across multiple processes. (Allows only one instance or process to be opened)

Why Mutex? Lock and Monitor provide Thread Synchronization inside same process, but Mutex help with cross process synchronization.

Mutex API:

- To create **unnamed mutex**:
Mutex mutex = new Mutex();
- To create **named mutex**:
Mutex mutex = new Mutex(false, "Global_MyMutex");
new Mutex (initiallyOwned, name, out createdNew)
- Mutex.WaitOne() - Acquire ownership and enter critical section if available
- Mutex.ReleaseMutex() - Leaves critical ownership and Release ownership
- Mutex.WaitOne(Timeout) - Timed acquisition attempt.
- Mutex.Dispose() - Clean OS resource

Eg: Only one Spotify instance, only one installer

Semaphore - Synchronization primitive that limits the number of threads or processes that can access a shared resource concurrently to a specified maximum.

Mutex allows only one instance to be opened. Mutex = 1 Slot

Semaphore gives access to multiple threads to a defined limit. Semaphore = N Slot

Semaphore API:

Semaphore sem = new Semaphore(initialCount,maxCount);

sem.WaitOne(); - if slot available take one, if not wait

sem.Release() - if resource usage finished slot returned

SemaphoreSlim - Lightweight fast in-memory gate. Same-process synchronization, async/await scenarios

SemaphoreSlim API:

SemaphoreSlim sem = new SemaphoreSlim();

sem.Wait(); - blocks thread, Sync version

sem.WaitAsync(); - non-blocking asyncing waiting, Async version

sem.Release(); - Return thread

Use cases of SemaphoreSlim:

DB connection limiting, Download concurrency limit, Background worker limits

ReadWriteLockSlim() - Synchronization primitive optimized for read-heavy workloads by allowing multiple concurrent readers but only one writer. (Many reader or one writer)

Why ReadWriterLockSlim? In lock() only one reader enters, all others wait, even tho only reading is done, which is taking too much time.

So **many reader** -> allow, **many writer** -> allow only one. For this only ReadWriterLockSlim is introduced.

APIs:

1. Read:

- rw.EnterReadLock();
- rw.ExitReadLock();

2. Write:

- `rw.EnterWriteLock();`
- `rw.ExitWriteLock();`

```
class Program
{
    static ReaderWriterLockSlim rw = new ReaderWriterLockSlim();
    static int data = 100;
    static void Reader()
    {
        rw.EnterReadLock();
        try {
            Console.WriteLine($"{Thread.CurrentThread.ManagedThreadId} data is: {data}");
            Thread.Sleep(2000);
        }
        finally {
            rw.ExitReadLock();
        }
    }
    static void Writer()
    {
        rw.EnterWriteLock();
        try {
            data++;
            Console.WriteLine($"Writer updated data to {data}");
            Thread.Sleep(2000);
        }
        finally {
            rw.ExitWriteLock();
        }
    }
    static void Main(string[] args)
    {
        new Thread(Reader).Start();
        new Thread(Reader).Start();
        new Thread(Writer).Start();
        new Thread(Writer).Start();
        new Thread(Reader).Start();
        new Thread(Writer).Start();
        Console.ReadLine();
    }
}
```

3. Upgradable read mode:

- `rw.EnterUpgradeableReadLock();` enter as reader, might change to writer later
- `rw.ExitUpgradeableReadLock();`

```

static void GetData(int id)
{
    rw.EnterUpgradeableReadLock();
    try
    {
        Console.WriteLine($"{Thread.CurrentThread.ManagedThreadId} reading....");
        if (cache.ContainsKey(id))
        {
            Console.WriteLine($"Found id: {id}");
        }
        else
        {
            Console.WriteLine($"Id {id} missing");
            rw.EnterWriteLock();
            try
            {
                Console.WriteLine($"{Thread.CurrentThread.ManagedThreadId} is writing.....");
                cache[id] = $"Watermelon";
            }
            finally
            {
                rw.ExitWriteLock();
            }
        }
        Console.WriteLine("The data read");
        foreach (var v in cache)
        {
            Console.WriteLine($"{v.Key} {v.Value}");
        }
    }
    finally
    {
        rw.ExitUpgradeableReadLock();
    }
}

```

Deadlock - Occurs when threads wait on each other indefinitely, so nobody can proceed

Eg:

Thread1 → owns LockA, wants LockB

Thread2 → owns LockB, wants LockA

Condition for deadlock:

1. Mutual Exclusion - Resource cannot be shared.
2. Hold and Wait - Thread holds resource while waiting for another.
3. No preemption - Cannot forcibly steal lock.
4. Circular wait - Cycle exists.

How to prevent?

Lock Ordering:

Bad: Lock 1 - lock a, lock b
Lock 2 - lock b, lock a

Good: Lock 1 - lock a, lock b
Lock 2 - lock a, lock b

Timeouts :

Instead of wait forever, wait for few seconds, if unavailable give up.
Eg: Monitor.TryEnter(), Mutex.WaitOne(timeout)

Starvation - Occurs when a thread keeps waiting indefinitely because other threads continuously get the resource first.

Priority Starvation

t1.Priority = ThreadPriority.Highest;
t2.Priority = ThreadPriority.Lowest;

Memory Barrier - Synchronization mechanism that restricts CPU/compiler reordering and improves visibility between threads.

ManualResetEvent - Signaling primitive that lets threads WAIT until another thread SIGNALS them

Heavier OS - Based signaling

Stays open until Reset is given manually

Key APIs:

ManualResetEvent mre = new ManualResetEvent(false);

mre.WaitOne() - Waits until signal received

mre.Set() - Signal event

mre.Reset() - Return to waiting state

ManualResetEventSlim - Lightweight same-process version

Key APIs:

ManualResetEventSlim mres = new ManualResetEventSlim(false);

mres.Wait() - Waits until signal received

mres.Set() - Signal event
mres.Reset() - Return to waiting state

Set()
↓
OPEN gate
↓
ALL waiting threads proceed
↓
Gate stays open

AutoResetEvent - Signaling primitive that releases ONE waiting thread per signal and automatically resets afterward.

Key APIs:

```
AutoResetEvent are = new AutoResetEvent(false);  
are.Set();  
are.WaitOne();
```

Set()
↓
Wake ONE thread
↓
Automatically close gate

CountdownEvent - Synchronization primitive that blocks until a specified number of operations complete.

Key APIs:

```
CountdownEvent cde = new CountdownEvent(3);  
cde.Signal(); - One task completed  
cde.Wait(); - Wait until count reaches zero  
Cde.AddCount() - Dynamically add more work
```

```
static CountdownEvent cde = new CountdownEvent(4); //Need 4 completion signals. number of thread and the number of countdownevent must be equal
static void Work()
{
    Console.WriteLine($"{Thread.CurrentThread.ManagedThreadId} is waiting.....");
    Thread.Sleep(2000);
    Console.WriteLine($"{Thread.CurrentThread.ManagedThreadId} is working.....");

    cde.Signal();
}
static void Main(string[] args)
{
    new Thread(Work).Start();
    new Thread(Work).Start();
    new Thread(Work).Start();
    new Thread(Work).Start();

    Console.WriteLine("Main thread started");

    cde.Wait();
    Console.WriteLine("All workds finished");
    Console.ReadLine();
}
```


DAY 4 - BackgroundWorker

4	BackgroundWorker (<i>legacy</i>)	<code>BackgroundWorker</code> class; <code>DoWork</code> , <code>ProgressChanged</code> , <code>RunWorkerCompleted</code> events; <code>ReportProgress</code> ; <code>CancellationPending</code> ; Cross-thread exception handling; Migration to async/await + <code>IProgress<T></code>	One session max. Teach it because legacy WinForms/WPF code uses it heavily. Emphasize: don't write new code with this.
---	--	---	---

BackgroundWorker - .NET component that executes long-running work on a background thread, reports progress safely to the UI thread, supports cancellation, and notifies completion without freezing the user interface

- **DoWork** - Contains the actual background processing logic; runs on a worker thread ie background thread (Long running operation)
- **ReportProgress() + ProgressChanged**: Used to report progress from the worker thread and safely update UI controls on the UI thread.
- **RunWorkerCompleted** - Executes after the background work finishes; used for final UI updates, results, or error handling.
- **CancellationPending** - Checked inside `DoWork` to determine whether cancellation has been requested via **`CancelAsync()`**.
- **Modern Alternative** - `Task`, `async/await`, `CancellationToken`, and `IProgress<T>` are preferred for new applications; `BackgroundWorker` is mainly used in legacy WinForms/WPF projects.

For `BackgroundWorker` to support work and cancellation we must enable this:

```
backgroundWorker1.WorkerReportsProgress = true;
```

```
backgroundWorker1.WorkerSupportsCancellation = true;
```

```

private void button1_Click(object sender, EventArgs e)
{
    backgroundWorker1.RunWorkerAsync();
}

private void backgroundWorker1_DoWork(object sender, DoWorkEventArgs e)
{
    for(int i = 1; i <= 100; i++)
    {
        Thread.Sleep(500);
        backgroundWorker1.ReportProgress(i);
    }
    Thread.Sleep(2000);
}

private void backgroundWorker1_ProgressChanged(object sender, ProgressChangedEventArgs e)
{
    progressBar1.Value = e.ProgressPercentage;
    label1.Text = $"Progress: {e.ProgressPercentage} %";
}

private void backgroundWorker1_RunWorkerCompleted(object sender, RunWorkerCompletedEventArgs e)
{
    MessageBox.Show("Progress Completed");
}

```

Cross-Thread Exception - Occurs when a background thread directly accesses a UI control that belongs to the UI thread. BackgroundWorker avoids this by executing ProgressChanged and RunWorkerCompleted on the UI thread.

Migration to Async/await:

Drawback of BackgroundWorker:

- Event-based programming
- More boilerplate code
- Harder error handling
- Less readable

Modern approach

- Task
- Task.Run() → RunWorkerAsync()
- async/await → RunWorkerAwaited() [Code after await]
- Async method → DoWork()
- CancellationToken → CancelAsync
- IProgress<T> → ReportProgress()

DAY 5 Task Parallel Library

5	Task Parallel Library (TPL)	Task vs Thread ; Task.Run vs Task.Factory.StartNew ; TaskCreationOptions.LongRunning ; Continuations (ContinueWith); Task.WhenAll , Task.WhenAny , Task.WaitAll ; Cancellation (CancellationToken , CancellationTokenSource , linked tokens); Exception handling; AggregateException ; IProgress<T>	Bridge from BackgroundWorker. Show side-by-side equivalents.
---	-----------------------------	--	--

Thread - A Thread is an **actual OS-level** execution unit.

```
Thread t = new Thread(() =>{
    Console.WriteLine("Running");
});
t.Start();
```

Task - A Task represents a **logical unit of work** that will execute asynchronously.
Uses threadpool

```
Task t = new Task(() => {
    Console.WriteLine("Running");
});
```

Task.Run() - Simplified way to queue work to the ThreadPool. [Return type Task]

```
Task.Run(() =>{
    Console.WriteLine("Hello");
});
```

Task.Factory.StartNew() - Advanced task creation API providing more control over task behavior. [Return type Task<Task>]

```
Task.Factory.StartNew(() => {
    Console.WriteLine("Running");
});
```

TaskCreationOptions.LongRunning - Tells the Task Scheduler that the task is expected to run for a long time. Scheduler assigns a dedicated thread.

Eg:

```
Task.Factory.StartNew(() => {
    while(true) {
        Console.WriteLine("Monitoring...");
        Thread.Sleep(1000);
    }
},
TaskCreationOptions.LongRunning);
```

Continuations (ContinueWith) - A continuation is a task that automatically executes after another task completes.

Task.WhenAll() - Waits for **all** task to complete, all the task runs concurrently ie Parallel. Asynchronous and doesn't block thread, returns Task

```
Task t1 = Task.Delay(2000);
Task t2 = Task.Delay(1000);
Task t3 = Task.Delay(3000);

Task.WhenAll(t1, t2, t3).GetAwaiter().GetResult(); //prints after 3sec

Console.WriteLine("All task completed");

Task<int> t4 = Task.Run(() => 10);
Task<int> t5 = Task.Run(() => 20);
Task<int> t6 = Task.Run(() => 30);
int[] result = Task.WhenAll(t4, t5, t6).GetAwaiter().GetResult();

foreach(var v in result)
{
    Console.Write(v + " "); // 10 20 30
}

Console.ReadLine();
```

Task.WhenAny() - Waits for the **first** task to complete.

```
Task<int> task1 = Task.Run(() =>
{
    Thread.Sleep(2000);
    return 20;
});
Task<int> task2 = Task.Run(() =>
{
    Thread.Sleep(3000);
    return 30;
});
Task<int> completedTask = Task.WhenAny(task1, task2).GetAwaiter().GetResult();
Console.WriteLine(completedTask.Result);
```

Task.WaitAll() - Blocks the current thread until all tasks complete.
Synchronous and blocks thread, returns Void

CancellationToken & CancellationTokenSource

CancellationTokenSource - Responsible for requesting cancellation.

CancellationToken - Responsible for receiving and checking cancellation requests.

To create CancellationTokenSource:

```
CancellationTokenSource cts = new CancellationTokenSource();
```

To request Cancel:

```
cts.Cancel();
```

To create CancellationToken:

```
CancellationToken token = new CancellationToken();
```

To check Cancellation

```
token.IsCancellationRequested
```

Throws exception:

```
token.ThrowIfCancellationRequested();
```

Auto cancel:

```
cts.CancelAfter(4000);
```

Linked Tokens - Combine multiple CancellationTokens into a single token.

```
CancellationTokenSource userCTS = new CancellationTokenSource();
CancellationTokenSource timeoutCTS = new CancellationTokenSource();
CancellationTokenSource linkedCTS =
CancellationTokenSource.CreateLinkedTokenSource
( userCTS.Token, timeoutCTS.Token);
```

This is similar to OR condition:

If userCTS or timeoutCTS is cancelled then linkedCTS will also gets cancelled

Exception Handling in Task - Task exceptions are captured and stored within the Task. When **awaited**, the **original exception** is rethrown. When using **Wait()** or **Result**, exceptions are wrapped inside an **AggregateException**.

Using Await:

```
static void Main(string[] args)
{
    try
    {
        RunAsync().GetAwaiter().GetResult();
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
    Console.ReadLine();
}

static async Task RunAsync()
{
    await Task.Run(() =>
    {
        throw new Exception("Something went wrong");
    });
}
```

Using Wait:

```
static void Main(string[] args)
{
    Task task = Task.Run(() =>
    {
        throw new Exception("Something went wrong");
    });

    try
    {
        task.Wait();
    }
    catch(Exception ex)
    {
        Console.WriteLine(ex.Message);
    }

    Console.ReadLine();
}
```

IProgress<T> - Provides a way for background tasks to report progress to the caller.

Used to report progress from an asynchronous operation back to the caller in a thread-safe manner.

To create IProgress: `IProgress<int> progress`

Implementation: `new Progress(() => { } );`

Report: `Progress.Report(i);`

Background Task → `progress.Report(50)` → `IProgress<int>` → UI Updated

DAY 6 - Async / Await

6	async / await	The state machine concept; <code>Task</code>, <code>Task<T></code>, <code>ValueTask<T></code>; <code>SynchronizationContext</code> (and WPF's); <code>ConfigureAwait(false)</code> — when and why; Pitfalls: <code>async void</code>, <code>.Result</code> / <code>.Wait()</code> deadlocks, fire-and-forget done wrong, missing <code>await</code>; Async in constructors / properties / <code>IAsyncDisposable</code>; <code>Task.Delay</code> vs <code>Thread.Sleep</code>	Allocate 2 sessions — this is where interns trip up most. Heavy hands-on.
---	---------------	---	--

Async - Modifier that marks a method as asynchronous (async doesn't create new thread)

Await - Operator used to wait non-blocking for an async operation to complete

```
async Task WorkAsync()
{
    await Task.Delay(1000);
}
```

Blocking -> `Thread.Sleep(3000);` - method waiting, thread blocked, no other work is done

Await -> `await Task.Delay(3000);` - method paused, thread released, other work is done

State machine compiler - When a method contains “await”, the compiler converts it into a state machine that can pause and resume execution.

Eg:

```
async Task Demo()
{
    Console.WriteLine("A");
    await Task.Delay(1000);
    Console.WriteLine("B");
}
```


State machine

```
if(state == 0) {  
    Console.WriteLine("A");  
    start delay;  
    state = 1;  
    return;  
}  
if(state == 1) {  
    Console.WriteLine("B");  
}
```

Task - An async operation. An object representing work that may complete in the future.

Eg:

```
async Task WorkAsync() {  
    await Task.Delay(1000); }  
}
```

Operation started -> Still running -> will finish later

Task<T> - An asynchronous operation that returns a value of type T.

```
async Task<int> GetAgeAsync() {  
    await Task.Delay(1000);  
    return 22; }  
}
```

Return Task<int> ie integer value

Void → no result, Task → async no result, int → result, Task<int> → async result.

ValueTask<T> - A lightweight alternative to **Task<T>** used to reduce allocations when results are often immediately available.

SynchronizationContext - While using async/await in UI framework like WPF, Winforms there's a issue where only UI thread can update the UI controls.

- UI Thread -> label.Text = "Done";
- Worker Thread -> label.Text = "Done"; ---- not allowed, causes exception and application crashes (Cross-thread operation exception)

SynchronizationContext is an abstraction that controls **where asynchronous continuations run after an await.**

SynchronizationContext.Current();

WPF → DispatcherSynchronizationContext

WinForms → WindowsFormsSynchronizationContext

Console → null / usually none

Await:

await Task.Delay(2000); -> **capture context + resume there**

ConfigureAwait:

await Task.Delay(1000).ConfigureAwait(false); -> **don't capture context + resume anywhere**

ConfigureAwait - Don't need original thread/context? - await task.ConfigureAwait(false)

Where do we use ConfigureAwait:

Library code, Backend / service code, API calls / DB calls, File I/O operations etc.,

Don't use ConfigureAwait(false) - WPF UI code, WinForms UI code, When updating controls (label, button, textbox) after await, When continuation needs original context/thread, UI event handlers (usually)

.Result() / **.Wait()** - These block the thread, using this causes UI thread block, deadlock in WPF/Winform, reduce scalability

Instead of : int value = await GetDataAsync(); int value = GetDataAsync().Result();

Instead of : await WorkAsync(); WorkAsync().Wait();

Await - Calling async method without awaiting it.

Fire-and-Forget - This is an async programming pattern where an asynchronous operation is **started but not awaited or monitored for completion**.

Eg: _ = WorkAsync(); or WorkAsync();

Async Constructors - C# doesn't allow the constructor to be async

Eg:

```
Class User {  
    public async User(){  
        await Task.Delay(2000);  
    }  
}
```

```

        Console.WriteLine("Loaded");
    }
} -----> creates compiler error, C# doesn't allow this

```

INSTEAD:

```

Class User{
    public string Data{get; private set;}
    public User() { }
    public static async Task<User> CreateAsync() {
        var service = new User();
        await service.LoadAsync();
        return service;
    }
    private async Task LoadAsync(){
        await Thread.Sleep(2000);
        Data = "Data not loaded"
    }
}
Class Program {
    static void Main(String[] args){
        User service = new User.CreateAsync().GetAwaiter().GetResult();
        Console.WriteLine(service.Data);
    }
}

```

Output : Data not loaded

Async Properties - An Async Property means a property that performs asynchronous work using **await** inside getter/setter.

Eg:

```

Class User{
    public async string Name{
        get {
            await Task.Delay(2000);
            return "Samyu";
        }
    }
}
} -----> creates compiler error, C# doesn't allow this

```

INSTEAD:

```
Class User {
    public async Task<string> GetNameAsync() {
        await Task.Delay(2000);
        return "Sam";
    }
}
Class Program {
    static void Main(string[] args) {
        User user = new User();
        string name = user.GetNameAsync().GetAwaiter().GetResult();
        Console.WriteLine(name);
    }
}
```

Output: Sam

IAsyncDisposable - is an **interface** that allows resources to be released asynchronously using **DisposeAsync()**.

Interface ->

```
public interface IAsyncDisposable {
    ValueTask DisposeAsync();
}
```

IDisposable - is an **interface** used to release unmanaged resources explicitly when they are no longer needed. Using

Create an Interface:

```
public interface IDisposable {
    void Dispose();
}
Class MyResource : IDisposable {
    public void Dispose() {
        Console.WriteLine("Task Completed");
    }
}
```

```

Class Program {
    using( var resource = new MyResource() ) {
        Console.WriteLine("Working.....");
    }
}

```

Dispose → synchronous process, since it can't have await Microsoft introduced `IDisposable`

IDisposable is used when a class owns resources that require **asynchronous cleanup**. Eg: Database, Network stream, HTTP Connection etc.,

Feature	<code>IDisposable</code>	<code>IDisposable</code>
Method	<code>Dispose()</code>	<code>DisposeAsync()</code>
Cleanup	Sync	Async
Return Type	void	ValueTask
Keyword	using	await using
Supports await	✗	✓

Task.Delay() vs Thread.Sleep()

Thread.Sleep() - Blocks the current thread for the specified duration.

Thread stops working, Thread cannot do anything else

Eg: testing, pausing dedicated worker thread

Task.Delay() - Creates a non-blocking asynchronous delay.

Method pauses, Thread becomes available

Eg: Async methods, API calls, UI Application

```

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Thread.Sleep");
        Console.WriteLine($"{Thread.CurrentThread.ManagedThreadId}");
        Thread.Sleep(2000);
        Console.WriteLine($"{Thread.CurrentThread.ManagedThreadId}");

        Console.WriteLine("\nTask.Delay");
        RunTask().GetAwaiter().GetResult();

        Console.ReadLine();
    }

    static async Task RunTask()
    {
        Console.WriteLine($"{Thread.CurrentThread.ManagedThreadId}");
        await Task.Delay(2000);
        Console.WriteLine($"{Thread.CurrentThread.ManagedThreadId}");
    }
}

```

For Thread.Sleep() → output will be like: 1 1

For Task.Delay() → output will be like: 1 4